

SQL Skill Dossier

Contents

1	Scope	2
1.1	Purpose	2
1.2	Primary Reference DBMS	2
1.3	What this dossier includes	2
1.3.1	Core SQL querying (baseline to advanced)	2
1.3.2	Relational design essentials (SQL-engineer level)	2
1.3.3	Performance & tuning (senior differentiator)	2
1.3.4	Transactions & concurrency (must for senior)	2
1.3.5	Production practices (SQL-adjacent, not DBA)	3
1.4	What this dossier intentionally excludes	3
2	Skill Ladder	4
2.1	Junior (Foundations)	4
2.2	Mid (Productive Professional)	4
2.3	Senior (Engineering Mastery)	5
2.4	Expert (Deep Reliability + Optimization + Strategy)	6
3	Concept Map / Index (Reading Order)	7
3.1	Core Querying Foundations (J → M)	7
3.1.1	SELECT Essentials	7
3.1.2	Joins & Relational Thinking	7
3.1.3	Aggregation & Grouping	7
3.1.4	Subqueries & Set Operations	7
3.2	Advanced Querying (M → S)	7
3.2.1	CTEs (WITH)	7
3.2.2	Window Functions	7
3.2.3	Data Types & Built-in Functions (Practical Core)	7
3.3	Relational Design Essentials (J → S)	8
3.3.1	Keys, Constraints, and Integrity	8
3.3.2	Normalization & Modeling Tradeoffs	8
3.4	Performance & Tuning (M → E)	8
3.4.1	Indexing Fundamentals	8
3.4.2	Query Plans (EXPLAIN)	8
3.4.3	Query Optimization Patterns	8
3.5	Transactions, Concurrency, and Production (S → E)	8
3.5.1	Transactions & Isolation	8
3.5.2	Locking & Deadlocks (Conceptual + Practical)	8
3.5.3	Production SQL Essentials	9

1 Scope

1.1 Purpose

This dossier is designed to help you master SQL as a professional skill from Junior → Mid → Senior → Expert, with a focus on conceptual understanding, decision-making, and real-world correctness (not tool tutorials). It is also interview-ready because you'll be able to explain concepts clearly.

1.2 Primary Reference DBMS

PostgreSQL is the reference dialect and feature set. It will include short “Cross-DB Notes” only when differences matter (MySQL and SQL Server).

1.3 What this dossier includes

1.3.1 Core SQL querying (baseline to advanced)

- SELECT fundamentals: filtering, ordering, limiting
- Joins: inner/left/right/full, anti-joins
- Aggregations: GROUP BY, HAVING, grouping pitfalls
- Subqueries and set operations: IN/EXISTS, UNION/INTERSECT/EXCEPT
- CTEs (WITH): readability and recursion (conceptual)
- Window functions: ranking, running totals, partitions (high-value)

1.3.2 Relational design essentials (SQL-engineer level)

- Relational model fundamentals: keys, relationships, cardinality
- Normalization basics (1NF–3NF) and when to denormalize
- Constraints: primary key, foreign key, unique, check, not null
- Data types: conceptual use, correctness and tradeoffs

1.3.3 Performance & tuning (senior differentiator)

- Query plans (EXPLAIN) at a practical conceptual level
- Index fundamentals: selectivity, composite indexes, covering/index-only concepts
- Common performance anti-patterns: N+1, missing predicates, bad joins, over-fetching
- Reasoning about cost: scans vs index lookups, cardinality intuition

1.3.4 Transactions & concurrency (must for senior)

- ACID concepts applied to real systems
- Isolation levels (practical understanding)
- Locking and deadlocks (conceptual + what causes them)
- Idempotency and safe retries (how SQL behavior impacts application behavior)

1.3.5 Production practices (SQL-adjacent, not DBA)

- Schema migrations (conceptual): backward compatibility, zero-downtime changes (high-level)
- Security essentials: SQL injection principles, least privilege, roles (conceptual)

1.4 What this dossier intentionally excludes

- Deep DBA topics: replication, HA, backups, server tuning, vacuum internals
- Deep planner internals research
- Stored procedures / triggers as a main skill (we may mention them briefly)
- Vendor-specific deep dives beyond essential differences

2 Skill Ladder

Use this ladder to self-assess your current level. A good rule: you “own” a level when you can explain it clearly and apply it consistently in real scenarios.

2.1 Junior (Foundations)

You can:

- Write correct queries using:
 - SELECT, WHERE, ORDER BY, LIMIT/OFFSET
 - basic JOINS (mostly INNER and LEFT)
 - GROUP BY + common aggregates (COUNT, SUM, AVG, MIN, MAX)
- Understand core relational concepts:
 - tables/rows/columns
 - primary keys vs foreign keys (at a basic level)
 - what duplicates are and why they appear
- Handle NULLs reasonably:
 - understand that NULL is not equal to anything
 - use IS NULL / IS NOT NULL
- Do safe data changes:
 - INSERT, UPDATE, DELETE with correct filtering (avoid “update all rows”)
- Use basic subqueries and IN (with awareness that NULL can affect results)

Signals you’re Junior+:

- You rarely break queries with syntax errors.
- You can explain what a query does line-by-line.

2.2 Mid (Productive Professional)

You can:

- Use advanced querying patterns:
 - EXISTS vs IN (and why EXISTS is often safer)
 - anti-joins (NOT EXISTS) for “find missing” cases
 - set operations (UNION, INTERSECT, EXCEPT) when appropriate
- Build reliable aggregates:
 - correct use of HAVING
 - avoid grouping mistakes (selecting non-grouped columns)
 - handle duplicates and join multiplicity intentionally
- Use CTEs (WITH) for readability and decomposition.

- Use window functions (basic):
 - ROW_NUMBER, RANK, DENSE_RANK
 - SUM(...) OVER (PARTITION BY ...) for running totals
- Understand indexing conceptually:
 - what an index is, how it speeds lookups
 - why indexes can slow down writes
 - basic idea of composite indexes
- Understand transactions at a working level:
 - why transactions exist
 - commit/rollback
 - what “read committed” generally means (conceptually)

Signals you’re Mid+:

- You can design queries that stay correct as data grows.
- You can explain why a query returns too many rows (join multiplicity).

2.3 Senior (Engineering Mastery)

You can:

- Diagnose and improve query performance using query plans (conceptual + practical reading):
 - explain EXPLAIN output at a high level (scans, joins, estimated rows)
 - reason about selectivity and cardinality
 - identify missing/wrong indexes and rewrite queries when needed
- Design schemas with integrity and tradeoffs:
 - normalization to 3NF and when to denormalize
 - choose constraints intentionally (PK/FK/unique/check)
 - understand how schema design impacts query complexity/performance
- Handle concurrency and correctness:
 - isolation levels and anomalies (dirty/non-repeatable/phantom reads) at a practical level
 - understand locking and how deadlocks can happen
 - write SQL that is safe under concurrent updates (conceptually)
- Build reliable production queries:
 - pagination patterns and their tradeoffs (offset vs keyset/cursor)
 - avoid N+1 patterns across app ↔ DB boundaries
 - reason about idempotency and safe retries (application + DB)

- Communicate clearly:
 - explain tradeoffs to developers and stakeholders (e.g., “why this query is slow”, “why this schema is safer”)

Signals you’re Senior+:

- You can look at a slow query and propose 2–3 plausible causes and next steps (plan, indexes, rewrite).
- You can design data models that reduce bugs through constraints, not just application logic.

2.4 Expert (Deep Reliability + Optimization + Strategy)

You can:

- Perform advanced performance reasoning:
 - choose composite/covering indexes strategically
 - understand join strategies (nested loop vs hash join) conceptually
 - estimate which part of a query is the bottleneck
- Design for scale:
 - partitioning concepts (why/when)
 - large-data pagination strategies at scale
 - write queries that remain stable and predictable with huge datasets
- Lead DB-related engineering decisions:
 - define SQL standards for teams (patterns, review checklist)
 - plan safe schema evolution (backward-compatible, zero-downtime concepts)
 - advise on tradeoffs: OLTP vs analytics patterns (conceptually)
- Security and governance awareness:
 - least privilege and access control concepts
 - SQL injection threat model and prevention principles (from both SQL and application sides)

Signals you’re Expert+:

- You can guide others through performance incidents and propose durable fixes.
- You can anticipate failure modes (locks, load spikes, bad plans) and design around them.

3 Concept Map / Index (Reading Order)

This is the recommended reading order for mastering SQL efficiently. It's organized from core correctness → advanced querying → design → performance → production.

3.1 Core Querying Foundations (J → M)

3.1.1 SELECT Essentials

- SELECT, FROM, WHERE, ORDER BY, LIMIT/OFFSET
- filtering logic, operator precedence
- NULL semantics (IS NULL, three-valued logic basics)

3.1.2 Joins & Relational Thinking

- INNER, LEFT, RIGHT, FULL
- join conditions, join keys
- duplicates and join multiplicity (why row counts “explode”)

3.1.3 Aggregation & Grouping

- GROUP BY, aggregates, HAVING
- grouping correctness rules
- distinct vs group, counting pitfalls

3.1.4 Subqueries & Set Operations

- scalar vs table subqueries
- EXISTS vs IN
- UNION / UNION ALL, INTERSECT, EXCEPT
- anti-joins (NOT EXISTS) patterns

3.2 Advanced Querying (M → S)

3.2.1 CTEs (WITH)

- readability and decomposition
- recursive CTEs (conceptual use cases)

3.2.2 Window Functions

- ranking: ROW_NUMBER, RANK, DENSE_RANK
- analytic windows: SUM/AVG OVER, moving totals
- partitions and frames (conceptual)

3.2.3 Data Types & Built-in Functions (Practical Core)

- numeric/text/date-time types (conceptual correctness)
- casting, formatting, common pitfalls (timezone awareness)

3.3 Relational Design Essentials ($J \rightarrow S$)

3.3.1 Keys, Constraints, and Integrity

- primary keys, foreign keys, unique, check, not null
- why constraints prevent bugs

3.3.2 Normalization & Modeling Tradeoffs

- 1NF–3NF basics
- when and why to denormalize
- modeling one-to-many / many-to-many

3.4 Performance & Tuning ($M \rightarrow E$)

3.4.1 Indexing Fundamentals

- what indexes do, selectivity
- composite indexes (order matters conceptually)
- write overhead tradeoffs

3.4.2 Query Plans (EXPLAIN)

- scans vs index scans
- join strategies (high-level)
- cardinality estimation intuition

3.4.3 Query Optimization Patterns

- avoid unnecessary work (filter early)
- reduce row explosion (join strategy)
- remove N+1 at app/DB boundary
- correct pagination strategies (offset vs keyset)

3.5 Transactions, Concurrency, and Production ($S \rightarrow E$)

3.5.1 Transactions & Isolation

- ACID applied
- isolation levels and anomalies (practical)
- safe retries and idempotency

3.5.2 Locking & Deadlocks (Conceptual + Practical)

- common causes
- how to reduce risk via query and transaction design

3.5.3 Production SQL Essentials

- schema migrations (backward compatibility concepts)
- zero-downtime change mindset (high-level)
- security essentials: SQL injection, least privilege (conceptual)